

Отчет к Лабораторной работе N°2 "Интерполяция"

Шуман Владислав Олегович

3 курс, 4 группа

Вариант 2А

[Github](#)

Время последнего изменения: 23.03.2026

Часть из электронных конспектов, которую я использовал: [тут](#)

Условие

2. Интерполяционный многочлен II

- Написать программу, которая строит для данной функции интерполяционный многочлен с кратными узлами P_n произвольной степени по соответствующим вашему варианту узлам (в форме Ньютона), и позволяет вычислять значения этих многочленов в указанной точке. Вычисление $P_n(x)$ следует осуществлять по схеме Горнера.
- Провести вычислительный эксперимент.
 - С помощью написанной программы построить совмещенную логарифмическую диаграмму сходимости интерполяционного процесса для функции f . По оси абсцисс на диаграмме изменяется степень многочлена (например, от 1 до 101 с шагом 5, или что-то типа этого), по оси ординат – норма погрешности $\|r_n\| = \max_i |f(\xi_i) - P_n(\xi_i)|$. Здесь $\{\xi_i\}$ – равномерная сетка из 1013 узлов на отрезке интерполяции.
 - Построенной диаграмме примерно определить степень M интерполяционного многочлена, гарантирующую погрешность ε .
 - Для степеней $n = N$ (см. пункт 1.1) и $n = M$ построить графики остатков интерполирования $r_n(x) = f(x) - P_n(x)$.
- **Составить отчет**, который содержит
 - Ответы на вопросы:
 1. Как ведет себя погрешность интерполяции с ростом степени интерполяционного многочлена? Почему?
 2. Какую минимальную погрешность интерполяции можно достигнуть с помощью вашей программы? Почему?
 - Графики и диаграммы (требования см. в конце документа)
 - Код программы

Варианты

A. $f(x) = \cos 2x$, $\varepsilon = 10^{-3}$, $[a, b] = [-1, 1]$. Чебышевские узлы интерполяции, кратность каждого узла равна 2.

пункт 1.1

- Оценить степень N интерполяционного многочлена по чебышевским узлам, необходимую для приближения функции f на отрезке $[a, b]$ с точностью не менее ε .

Решение

- Оценить степень N интерполяционного многочлена по чебышевским узлам, необходимую для приближения функции f на отрезке $[a, b]$ с точностью не менее ε .

Эти формулы можно записать проще, рассмотрев новую последовательность узлов $\{\xi_i\}_{i=0}^{N-1}$, полученную из $\{x_i\}_{i=0}^n$ повторением i -го узла m_i раз:

$$\Xi = \{\xi_i\}_{i=0}^{N-1} = \underbrace{\{x_0, \dots, x_0\}}_{m_0}, \underbrace{\{x_1, \dots, x_1\}}_{m_1}, \dots, \underbrace{\{x_n, \dots, x_n\}}_{m_n}. \quad (3.12)$$

$$\varphi_i(x) = \Omega_i(x) = \prod_{j=0}^{i-1} (x - \xi_j).$$

$$\|\omega_{n+1}\| = 2 \left(\frac{b-a}{4} \right)^{n+1}.$$

Теорема 3.1. Пусть f — интерполируемая функция, H_{N-1} — ИМ Эрмита по расширенному множеству узлов (3.12). Тогда

$$r_{N-1}(x) = f(x) - H_{N-1}(x) = f[\xi_0, \dots, \xi_{N-1}, x] \Omega_N(x), \quad (3.17)$$

где $\Omega_N(x)$ вычисляется по (3.13).

Доказательство проводится аналогично теореме 2.1.

Теорема 3.2. Пусть $f \in C^{(N)}[a, b]$, интерполируемая функция, H_{N-1} — ИМ Эрмита по множеству узлов $\{x_i\}_{i=0}^n$ с кратностями m_i , $\sum_{i=0}^n m_i = N$. Тогда $\forall x \in [a, b] \exists \xi \in (\underline{x}, \bar{x})$, $(\underline{x} = \min\{x_0, \dots, x_n, x\}, \bar{x} = \max\{x_0, \dots, x_n, x\})$, такое, что

$$r_{N-1}(x) = f(x) - H_{N-1}(x) = \frac{f^{(N)}(\xi)}{N!} \Omega_N(x). \quad (3.18)$$

Оценить степень N ЦМ по че-
бышевским узлам, неаппроксимиру-
емую приближением f на $[a, b]$
с точностью не менее ε .

$$\textcircled{a} \quad r_{k-1}(x) = \frac{f^{(k)}(\xi)}{k!} \Omega_k(x)$$

иногда $N = k - 1$

$$\|z\| \leq \frac{\|f^{(k)}(\xi)\|}{k!} \|\Omega_k(x)\|$$

$$\Omega_k = \prod_{j=0}^{k-1} (x - \xi_j)$$

$f(x) = \cos 2x$, $\varepsilon = 10^{-3}$, $[a, b] = [-\frac{\pi}{4}, \frac{\pi}{4}]$,
Чебышевские узлы интерполируют ε -
кратность Канторов 2:

так как ε кратность 2, то

$$\Omega_k = \prod_{i=0}^n (x - x_i)^2 = (w_{n+1}(x))^2$$

$$\text{т.о.} \quad \|\Omega_k\| = \|w_{n+1}(x)\|^2$$

$$\begin{aligned} \|w_{n+1}\| &= 2 \left(\frac{b-a}{4} \right)^{n+1} = \int_{[-\frac{\pi}{4}, \frac{\pi}{4}]} \\ &= 2 \left(\frac{2}{4} \right)^{n+1} = 2^{-n} \end{aligned}$$

$$T=0. \quad \|x_k\| \leq 2^{-2n}.$$

$$\|f^{(k)}\| :$$

$$f(x) = \cos 2x$$

$$f^{(1)}(x) = 2^1 \cdot -\sin 2x =$$

$$= 2^1 \cos\left(2x + \frac{1 \cdot \pi}{2}\right)$$

$$f^{(2)}(x) = 2^2 \cdot \cos 2x =$$

$$= 2^2 \cos\left(2x + \frac{2 \cdot \pi}{2}\right) =$$

$$= 2^2 \cos\left(2x + \frac{2 \cdot \pi}{2}\right)$$

$$f^{(k)} = 2^k \cos\left(2x + \frac{k \cdot \pi}{2}\right)$$

$$\|f^{(k)}\| = \|f^{(k)}\|_{C[a,b]} =$$

$$= \max_{x \in [a,b]} \left| 2^k \cos\left(2x + \frac{k \cdot \pi}{2}\right) \right| \leq 2^k.$$

$$\text{таким образом: } \|x_k\| \leq \frac{2^k}{k!} \cdot 2^{-2n} \leq \frac{2}{2}$$

$$\left[k \leq 2(n+1) \right] : \frac{2^k}{k!} \cdot 2^{-(k-2)} \leq 2$$

$$\frac{2^k}{k!} \cdot \frac{2^2}{2^k} = \frac{4}{k!} \leq 10^{-5};$$

$$k! \geq 4000 : \quad \begin{array}{l} 1! = 1 \\ 6! = 720 \\ 7! = 5040 \end{array}$$

т.к. наименьшее k такое, что $2^k \geq 4000$,
 $k = 8$

$$N = k - 1 = 7$$

III

Ответ: 7.

- Написать программу, которая строит для данной функции интерполяционный многочлен с кратными узлами P_n произвольной степени по соответствующим вашему варианту узлам (в форме Ньютона), и позволяет вычислять значения этих многочленов в указанной точке. Вычисление $P_n(x)$ следует осуществлять по схеме Горнера.

вычислить $P_n(x)$ по т. н. схеме Горнера

$$P_n(x) = \alpha_0 + (x - x_0)(\alpha_1 + (x - x_1)(\alpha_2 + \dots)). \quad (2.6)$$

$$\alpha_i = \left(y_i - \sum_{j=0}^{i-1} \alpha_j \omega_j(x_i) \right) / \omega_i(x_i), \quad i = \overline{0, n}. \quad (2.4)$$

Пусть $\{y_i\}_{i=0}^n$ являются значениями некоторой функции f в точках $\{x_i\}_{i=0}^n$ соответственно. Тогда каждый из коэффициентов α_i , определяемых по формулам (2.4) можно рассматривать как выражение, зависящее от функции f и узлов $x_0, \dots, x_i$. Обозначим это выражение

$$\alpha_i = f[x_0, \dots, x_i]. \quad (2.7)$$

Итак, по построению в базисе (3.13) матрица Φ из (3.3) будет иметь нижнетреугольный вид. Осталось вывести формулы для выражения α_i , которые (пока что формально) снова обозначим

$$\alpha_i = f[\xi_0, \dots, \xi_i]. \quad (3.14)$$

$$\Xi = \{\xi_i\}_{i=0}^{N-1} = \{\underbrace{x_0, \dots, x_0}_{m_0}, \underbrace{x_1, \dots, x_1}_{m_1}, \dots, \underbrace{x_n, \dots, x_n}_{m_n}\}.$$

$$\Xi = \{x_0, x_0, x_1, x_1, \dots, x_n, x_n\}.$$

Рассмотрим достаточно гладкую функцию f и набор вещественных чисел $\Xi = \{\xi_j\}_{j=0}^i$, среди которых возможны совпадения. Разделённой разностью называют выражение

$$f[\xi_0, \dots, \xi_i] = f[\Xi] = \begin{cases} \frac{f^{(i)}(\xi_0)}{i!}, & \text{если } \xi_0 = \xi_1 = \dots = \xi_i; \\ \frac{f[\Xi \setminus \xi_q] - f[\Xi \setminus \xi_p]}{\xi_p - \xi_q}, & \text{если } \exists \xi_p \neq \xi_q. \end{cases} \quad (3.15)$$

- Если $x_i \neq x_j$: $\frac{f(x_i) - f(x_j)}{x_i - x_j}$.
- Если кратны и при условии, что у нас кратность 2: $\frac{f'(x_i)}{1!}$.

3.3.3 Алгоритм построения ИМ Эрмита в форме Ньютона

Собирая всё вышеизложенное вместе, получаем, что

$$H_{N-1}(x) = \sum_{i=0}^{N-1} f[\xi_0, \dots, \xi_i] \Omega_i(x), \quad (3.16)$$

Из (4.9) имеем, что для $n \geq 1$ старший коэффициент T_n равен 2^{n-1} . Следовательно, в формуле (4.7) неизвестная константа E равна 2^{1-n} , и искомый многочлен ω_{n+1} имеет вид

$$\omega_{n+1}(x) = 2^{-n} T_{n+1}(x). \quad (4.10)$$

Такой многочлен, в силу понятных причин, называют *многочленом, наименее отклоняющимся от нуля на отрезке $[-1, 1]$* . Его корни по построению являются оптимальными узлами интерполяции. Они равны корням многочлена Чебышева T_{n+1} :

$$x_i = \cos \frac{\pi(2i+1)}{2n+2}, \quad i = \overline{0, n}. \quad (4.11)$$

Узлы (4.11) называются *чебышёвскими узлами*.

Корни этого многочлена (оптимальные узлы интерполяции на отрезке $[a, b]$), очевидно, получаются масштабированием узлов (4.11):

$$x_i = \frac{a+b}{2} + \frac{b-a}{2} \cos \frac{\pi(2i+1)}{2n+2}, \quad i = \overline{0, n}. \quad (4.14)$$

идут по определению (3.15). Приведём пример такой таблицы для случая $n = 2$, $m_0 = 2$, $m_1 = 3$, $m_2 = 1$.

$x_0, f(x_0)$	$f'(x_0)$				
$x_0, f(x_0)$	$f[x_0, x_0]$	$f[x_0, x_0, x_1]$			
	$f[x_0, x_1]$	$f[x_0, x_0, x_1, x_1]$			
$x_1, f(x_1)$	$f[x_0, x_1, x_1]$	$f[x_0, x_0, x_1, x_1, x_1]$	$f[x_0, x_0, x_1, x_1, x_1]$		
	$f'(x_1)$	$f[x_0, x_1, x_1, x_1]$	$f[x_0, x_0, x_1, x_1, x_1, x_2]$		
$x_1, f(x_1)$	$f''(x_1)/2!$	$f[x_0, x_1, x_1, x_1, x_2]$	$f[x_0, x_1, x_1, x_1, x_2]$		
	$f'(x_1)$	$f[x_1, x_1, x_1, x_2]$			
$x_1, f(x_1)$	$f[x_1, x_1, x_2]$				
	$f[x_1, x_2]$				
$x_2, f(x_2)$					

найдем n :

$$K=2(n+1)$$

$$8=2(n+1)$$

$$4=n+1$$

$$n=3$$

```
import numpy as np

# исходная функция
def f(x):
    return np.cos(2 * x)

# ее производная
def df(x):
    return -2 * np.sin(2 * x)

# получить узлы Чебышева
def get_chebyshev_nodes(
    n, # n+1 -- количество различных узлов
    a, b): # [a,b]
    indices = np.arange(n + 1) # массив от 0 до n
    nodes = np.cos((np.pi * (2 * indices + 1)) / (2 * n + 2)) # узлы
```

```

чебышева
    return 0.5 * (a + b) + 0.5 * (b - a) * nodes # масштабирование

# построение таблицы разделенных разностей для кратных узлов
def build_hermite_table(
    nodes, # уникальные узлы
    multiplicity=2): # кратность каждого узла. по условию 2

    # Xi
    xi = np.repeat(nodes, multiplicity)
    n_total = len(xi)

    # таблица разностей: n_total строк на n_total столбцов
    table = np.zeros((n_total, n_total))

    # первый столбец: значения функции в узлах
    table[:, 0] = f(xi)

    # заполняем остальное
    for j in range(1, n_total):
        for i in range(n_total - j):
            # совпадают ли узлы
            if abs(xi[i+j] - xi[i]) < 1e-12:
                table[i, j] = df(xi[i])
            else:
                table[i, j] = (table[i+1, j-1] - table[i, j-1]) /
(xi[i+j] - xi[i])

    return xi, table[0, :]

# схема Горнера
def hornering(x, xi, alphas):
    n = len(alphas)
    res = alphas[-1]
    for i in range(n - 2, -1, -1):
        res = res * (x - xi[i]) + alphas[i]
    return res

a, b = -1, 1
n = 3

# узлы
unique_nodes = get_chebyshev_nodes(n, a, b)

# таблица и альфы
xi_expanded, alphas = build_hermite_table(unique_nodes)

# пробуем что-то вычислить
x_test = 0.5
y_poly = hornering(x_test, xi_expanded, alphas)

```

```

y_true = f(x_test)

print(f"степень многочлена: {len(alphas)-1}")
print(f"значение в x={x_test}: {y_poly}")
print(f"точное значение: {y_true}")
print(f"ошибка: {abs(y_poly - y_true)}")

степень многочлена: 7
значение в x=0.5: 0.5402798767315129
точное значение: 0.5403023058681398
ошибка: 2.2429136626866963e-05

```

■ Провести вычислительный эксперимент.

- С помощью написанной программы построить совмещенную логарифмическую диаграмму сходимости интерполяционного процесса для функции f . По оси абсцисс на диаграмме изменяется степень многочлена (например, от 1 до 101 с шагом 5, или что-то типа этого), по оси ординат – норма погрешности $\|r_n\| = \max_i |f(\xi_i) - P_n(\xi_i)|$. Здесь $\{\xi_i\}$ – равномерная сетка из 1013 узлов на отрезке интерполяции.
- Построенной диаграмме примерно определить степень M интерполяционного многочлена, гарантирующую погрешность ε .
- Для степеней $n = N$ (см. пункт 1.1) и $n = M$ построить графики остатков интерполирования $r_n(x) = f(x) - P_n(x)$.
- С помощью написанной программы построить совмещенную логарифмическую диаграмму сходимости интерполяционного процесса для функции f . По оси абсцисс на диаграмме изменяется степень многочлена (например, от 1 до 101 с шагом 5, или что-то типа этого), по оси ординат – норма погрешности $\|r_n\| = \max_i |f(\xi_i) - P_n(\xi_i)|$. Здесь $\{\xi_i\}$ – равномерная сетка из 1013 узлов на отрезке интерполяции.

```

import matplotlib.pyplot as plt
import matplotlib.ticker as ticker

xi_control = np.linspace(a, b,
                        1013) # сетка из 1013 узлов
y_true_control = f(xi_control)

# результат
degrees = []
errors = []

print("start...")

for target_degree in range(1, 102, 2):
    # кол_узлов = (на_степень + 1) / 2
    n_nodes = (target_degree + 1) // 2

    # если n_nodes == 0

```



```

if n_nodes < 1:
    n_nodes = 1

# узлы
nodes = get_chebyshev_nodes(n_nodes - 1, a, b)

# строим многочлен
xi_expanded, alphas = build_hermite_table(nodes)

# фактическая степень
actual_degree = len(alphas) - 1

# ошибка
y_poly_control = np.array([hornering(x, xi_expanded, alphas) for x
in xi_control])
error_norm = np.max(np.abs(y_true_control - y_poly_control))

# сохраняем
degrees.append(actual_degree)
errors.append(error_norm)

print(f"степень: {actual_degree:3} | ошибка: {error_norm:.2e}")

# первая степень, где ошибка < 1e-3
N = next((d for d, e in zip(degrees, errors) if e < 1e-3), None)

# диаграмма
plt.figure(figsize=(12, 7))
plt.plot(degrees, errors, 'bo-', markersize=4)
plt.axhline(y=1e-3, color='r', linestyle='--', label='ε=10-3')

if N is not None:
    plt.axvline(x=N, color='g', linestyle=':', label=f'N={N}')

plt.yscale('log')
plt.xlabel('степень многочлена (n)')
plt.ylabel('норма погрешности ||rn||')
plt.title('диаграмма сходимости интерполяционного процесса')

ax = plt.gca()
ax.yaxis.set_major_locator(ticker.LogLocator(base=10.0, numticks=30))
ax.xaxis.set_major_locator(ticker.MultipleLocator(5))
ax.xaxis.set_minor_locator(ticker.MultipleLocator(1))

plt.grid(True, which="major", ls="-", alpha=0.6)
plt.grid(True, which="minor", ls=":", alpha=0.3)

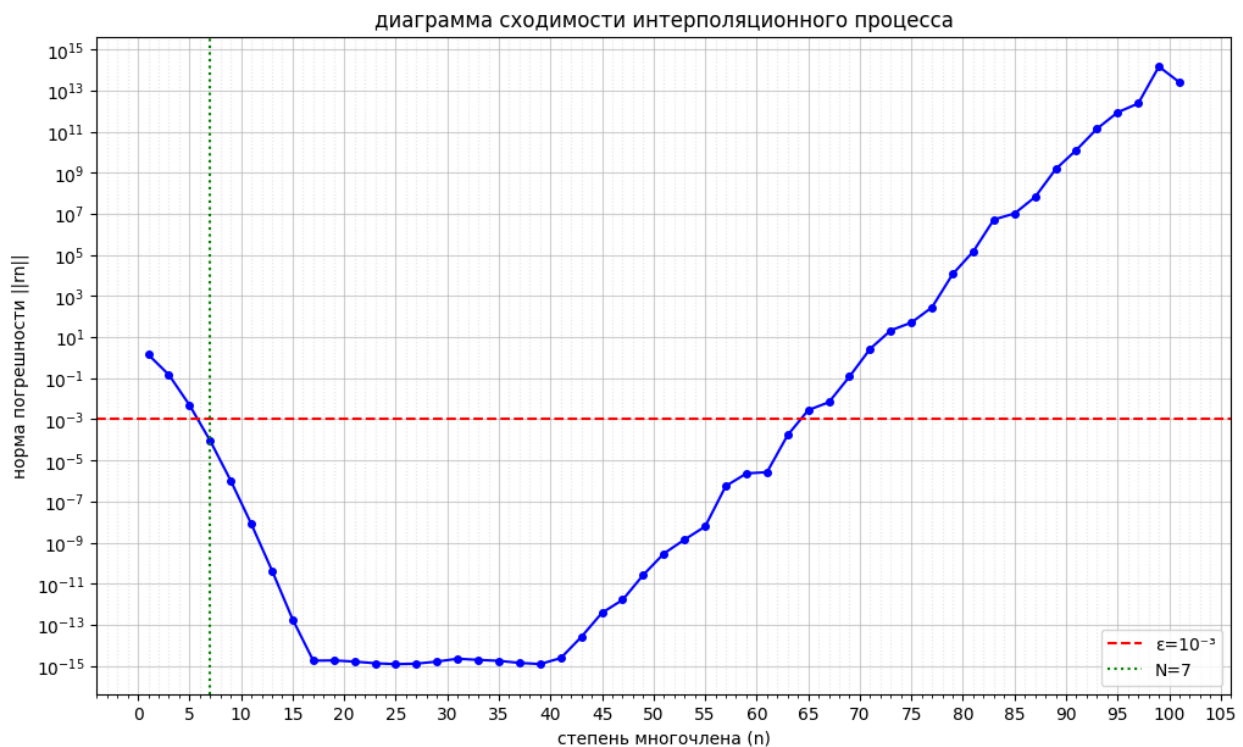
```

```
plt.legend()  
plt.show()
```

```
start...
```

степень:	1	ошибка:	1.42e+00
степень:	3	ошибка:	1.46e-01
степень:	5	ошибка:	4.90e-03
степень:	7	ошибка:	9.07e-05
степень:	9	ошибка:	1.02e-06
степень:	11	ошибка:	7.82e-09
степень:	13	ошибка:	4.32e-11
степень:	15	ошибка:	1.81e-13
степень:	17	ошибка:	1.78e-15
степень:	19	ошибка:	1.83e-15
степень:	21	ошибка:	1.61e-15
степень:	23	ошибка:	1.33e-15
степень:	25	ошибка:	1.21e-15
степень:	27	ошибка:	1.28e-15
степень:	29	ошибка:	1.61e-15
степень:	31	ошибка:	2.22e-15
степень:	33	ошибка:	1.94e-15
степень:	35	ошибка:	1.75e-15
степень:	37	ошибка:	1.39e-15
степень:	39	ошибка:	1.22e-15
степень:	41	ошибка:	2.39e-15
степень:	43	ошибка:	2.58e-14
степень:	45	ошибка:	3.82e-13
степень:	47	ошибка:	1.63e-12
степень:	49	ошибка:	2.66e-11
степень:	51	ошибка:	2.92e-10
степень:	53	ошибка:	1.39e-09
степень:	55	ошибка:	6.07e-09
степень:	57	ошибка:	5.62e-07
степень:	59	ошибка:	2.31e-06
степень:	61	ошибка:	2.68e-06
степень:	63	ошибка:	1.70e-04
степень:	65	ошибка:	2.81e-03
степень:	67	ошибка:	6.79e-03
степень:	69	ошибка:	1.21e-01
степень:	71	ошибка:	2.70e+00
степень:	73	ошибка:	2.13e+01
степень:	75	ошибка:	5.11e+01
степень:	77	ошибка:	2.84e+02
степень:	79	ошибка:	1.16e+04
степень:	81	ошибка:	1.48e+05
степень:	83	ошибка:	5.29e+06
степень:	85	ошибка:	1.04e+07
степень:	87	ошибка:	6.64e+07
степень:	89	ошибка:	1.51e+09
степень:	91	ошибка:	1.29e+10

степень:	93	ошибка:	1.40e+11
степень:	95	ошибка:	8.71e+11
степень:	97	ошибка:	2.38e+12
степень:	99	ошибка:	1.48e+14
степень:	101	ошибка:	2.52e+13



- Построенной диаграмме примерно определить степень M интерполяционного многочлена, гарантирующую погрешность ϵ .

$M=7$ (N) и $M=65$

- Для степеней $n = N$ (см. пункт 1.1) и $n = M$ построить графики остатков интерполирования $r_n(x) = f(x) - P_n(x)$.

```
# степени
degrees_to_plot = [7, 65]

plt.figure(figsize=(15, 6))

for i, n_val in enumerate(degrees_to_plot):
    # многочлен
    n_nodes = (n_val + 1) // 2
    nodes = get_chebyshev_nodes(n_nodes - 1, a, b)
    xi_exp, alphas = build_hermite_table(nodes)
```

```

# считаем значения
x_plot = np.linspace(a, b, 1000)
y_true_plot = f(x_plot)
y_poly_plot = np.array([hornering(x, xi_exp, alphas) for x in
x_plot])

# остаток
residual = y_true_plot - y_poly_plot

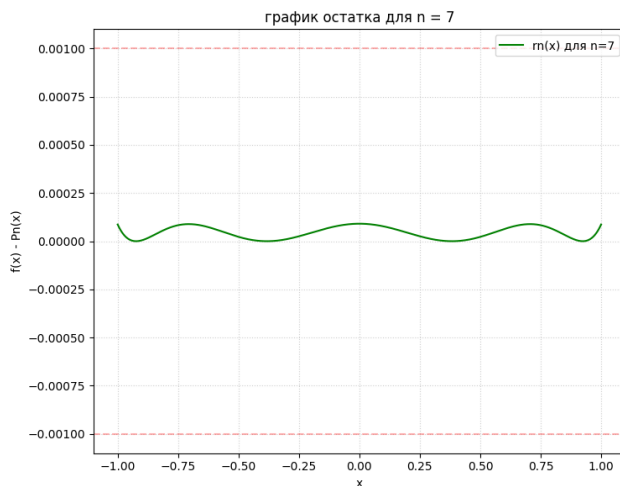
# рисуем
plt.subplot(1, 2, i + 1)
plt.plot(x_plot, residual, 'g-', label=f'rn(x) для n={n_val}')

plt.axhline(y=1e-3, color='r', linestyle='--', alpha=0.3)
plt.axhline(y=-1e-3, color='r', linestyle='--', alpha=0.3)

plt.title(f'график остатка для n = {n_val}')
plt.xlabel('x')
plt.ylabel('f(x) - Pn(x)')
plt.grid(True, ls=':', alpha=0.6)
plt.legend()

plt.tight_layout()
plt.show()

```



■ Составить отчет, который содержит

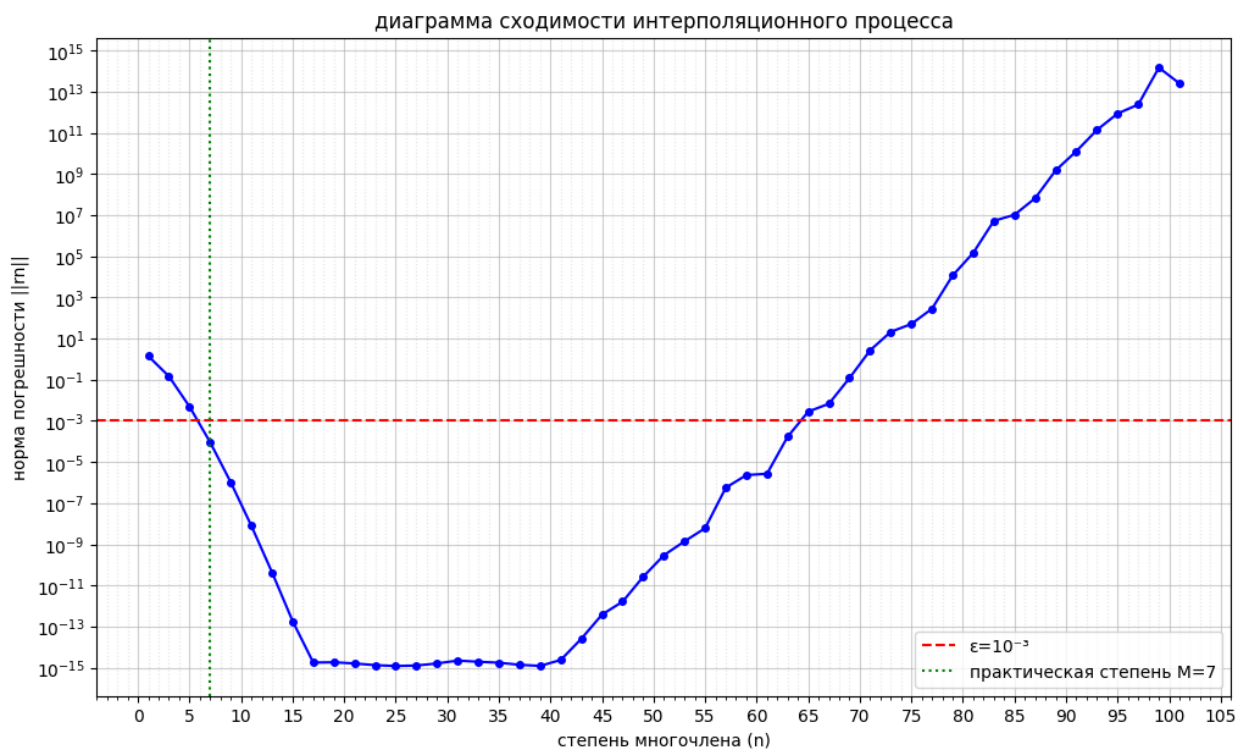
- Ответы на вопросы:
 1. Как ведет себя погрешность интерполяции с ростом степени интерполяционного многочлена? Почему?
 2. Какую минимальную погрешность интерполяции можно достигнуть с помощью вашей программы? Почему?
- Графики и диаграммы (требования см. в конце документа)
- Код программы


```
import sys
print(sys.float_info.epsilon)
print(sys.float_info.min)
```

```
2.220446049250313e-16
2.2250738585072014e-308
```

Как ведет себя погрешность интерполяции с ростом степени интерполяционного многочлена?

еще раз посмотрим на график



как мы видим, сначала погрешность стремительно падает, затем достигает "низа" (так как мы используем float64, то там $\epsilon \approx 2.22 \cdot 10^{-16}$ (фрагмент кода выше)), а далее, с некоторой степени, снова поднимается вверх

Почему?

Кроме этого, интерполяция по равноотстоящим узлам обладает еще одним неприятным свойством: она плохо обусловлена, т. е. при достаточно большом количестве точек небольшие погрешности округления будут приводить к большим погрешностям. Значит, даже если для функции $\cos$ интерполяция по равноотстоящим узлам сходится *теоретически* по теореме 5.6, на практике (в машинной арифметике) с ростом числа узлов интерполяционный процесс будет расходиться.

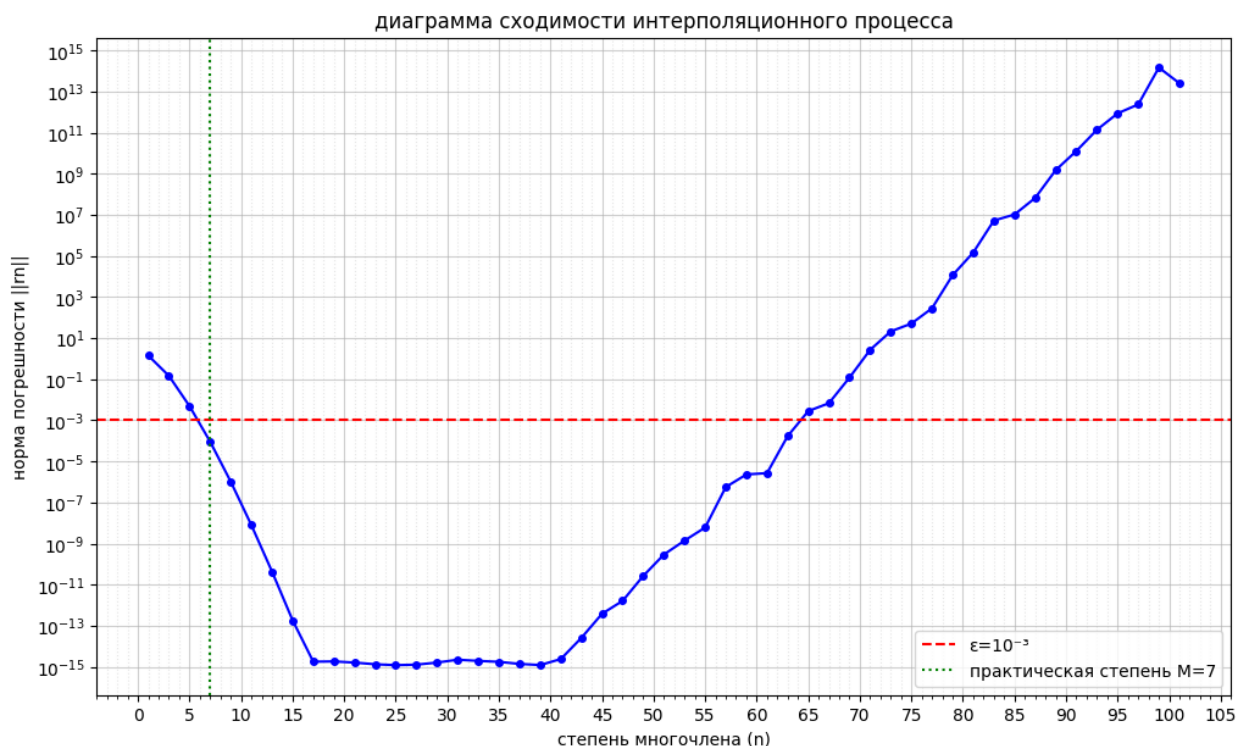
Более привлекательна интерполяция по чебышевским узлам.

Теорема 5.7. *Для любой абсолютно непрерывной на $[a, b]$ функции f интерполяционный процесс по чебышевским узлам равномерно сходится.*

процесс интерполяции при большом числе узлов становится плохо обусловленным. то есть, это значит, что малые погрешности (в моем случае с float64) приводят к большим ошибкам в результате.

в форме Ньютона это проявляется при вычислении разделенных разностей (из-за деления на малые разности близких узлов, погрешности округления накапливаются и вызывают расходимость на практике, несмотря на сходимость метода)

Какую минимальную погрешность интерполяции можно достигнуть с помощью вашей программы?



из диаграммы сходимости, минимальная норма погрешности составила 10^{-15} , что достигается в диапазоне от $n \approx 17$ до $n \approx 40$.

Почему?

связано с тем, что -- программа написана на Python с использованием NumPy, где типом данных для дробных чисел является `float64`. точность для которого составляет $\epsilon_{mach} \approx 2.22 \cdot 10^{-16}$.

в теории можно получить еще более точный результат (воспользоваться `mpmath`, например)